CS210: Backend Development Fundamentals

Lab 01 Report — Node.js & Express Server

AURORA SIDUS REPORT

Student Name:	Chinnala Sukumar	Registration No:	252U1R7059
Class & Section:	CSE FSD - 2B	Course:	Backend Development Fundamentals (CS210)

1. Objective

The objective of this laboratory is to initialize a Node.js project using `npm init -y`, configure and explain `package.json` metadata and scripts, build a native HTTP server utilizing Node.js's built-in `http` module, and develop an Express.js application implementing structured routing (`/`, `/about`, `/time`) and catch-all 404 error handling with correct HTTP status codes.

2. Tested Endpoints & Verified HTTP Status Codes

Method	Endpoint	Description	Expected Status	Actual Status	Test Result
GET	`/`	Returns welcome message	200 OK	**200 OK**	**PASSED**
GET	`/about`	Returns student profile details as JSON	200 OK	**200 OK**	**PASSED**
GET	`/time`	Returns current date and ISO timestamp	200 OK	**200 OK**	**PASSED**
GET	`/anything`	Catch-all handler for undefined paths	404 Not Found	**404 Not Found**	**PASSED**

3. Source Code: server-http.js (Built-in HTTP Module)

Demonstrates creating a basic server using only Node.js core libraries without third-party frameworks:

```
const http = require("http");

const server = http.createServer((request, response) => {
    response.writeHead(200, {
        "Content-Type": "text/plain"
    });

    response.end("Hello");
});

server.listen(3000, () => {
    console.log("HTTP server running at http://localhost:3000");
});
```

4. Source Code: server.js (Express Framework)

Express.js implementation configuring route handlers, JSON responses, dynamic environment port fallback, and a catch-all 404 middleware:

```
const express = require("express");

const app = express();
const PORT = process.env.PORT || 3000;

// GET /
app.get("/", (request, response) => {
    response.status(200).send("Welcome to my Lab 1 server!");
});

// GET /about
app.get("/about", (request, response) => {
    response.status(200).json({
        name: "Chinnala Sukumar",
        registrationNumber: "252U1R7059",
        section: "CSE FSD - 2B"
    });
});

// GET /time
app.get("/time", (request, response) => {
    response.status(200).json({
        currentDateTime: new Date()
    });
});

// Handle unknown routes
app.use((request, response) => {
    response.status(404).send("404 - Page Not Found");
});

// Start the server
app.listen(PORT, () => {
    console.log(`Express server running at http://localhost:${PORT}`);
});
```

Key Architectural Highlights

• **Port Configuration:** `const PORT = process.env.PORT || 3000;` enables local execution on port 3000 while seamlessly supporting cloud deployment environments.
• **Structured JSON:** `GET /about` outputs the student's exact details (`Chinnala Sukumar`, `252U1R7059`, `CSE FSD - 2B`) with `Content-Type: application/json`.
• **Dynamic Date:** `GET /time` utilizes `new Date()` to generate the live UTC ISO timestamp on every request.
• **404 Catch-All:** Registered as the final middleware function via `app.use()` to catch all undefined paths and return HTTP status 404 with `"404 - Page Not Found"`.

5. Genuine Execution Evidence (Browser Screenshots)

All screenshots below represent actual running server responses captured directly in Google Chrome:

1. Root Route (`GET /`) — 200 OK

2. About Route (`GET /about`) — 200 OK JSON



3. Time Route (`GET /time`) — 200 OK Timestamp



4. Unknown Route (`GET /anything`) — 404 Not Found

6. Viva Questions & Conceptual Explanations

Q1: What is the difference between package.json and node_modules?

package.json is a lightweight, human-readable configuration manifest that stores project metadata (name, version, scripts) and lists the names and semantic version ranges of installed dependencies. It is checked into version control. In contrast, **node_modules** is the physical directory on disk containing the actual extracted package files, binaries, and transitive dependencies installed by npm. It is machine-dependent, very large, and strictly excluded from Git via `.gitignore`.

Q2: What does app.listen() do?

`app.listen()` is an Express method that wraps Node's underlying `http.createServer()`. It binds the application to a designated TCP network port (e.g., 3000) and begins listening for incoming HTTP connections, directing each matching request to registered middleware and route handlers.

Q3: What do request and response represent?

The **request** (`req`) object encapsulates the incoming HTTP message sent by the client, exposing headers, URL path, query strings, parameters, and HTTP method. The **response** (`res`) object provides the server-side interface to formulate and transmit the HTTP response, including status codes (e.g. `res.status(200)`), headers, and payloads via `res.send()` or `res.json()`.

7. Problems Encountered & Real Solutions

1. Port 3000 Collision: Running `node server-http.js` and `node server.js` simultaneously produced an `EADDRINUSE` exception. *Resolution:* Explicitly terminated previous server instances before launching the next, and implemented `process.env.PORT || 3000` in `server.js` to support dynamic port assignment.

2. Headless Browser Screenshot Navigation on 404: Standard Chromium CLI flags (`--screenshot`) abort rendering when an HTTP 404 status is returned. *Resolution:* Connected directly to the Chrome DevTools Protocol (CDP) WebSocket endpoint to capture the authentic rendered browser response for the 404 error page.

8. Repository & Submission Summary

All code has been verified inside `lab-01/` with clean Git practices (excluding `node_modules/` and `.env`). All 4 required routes were tested and produced 100% genuine execution evidence. This report is compiled for upload to the Aurora SIDUS portal for CS210 Lab Report 1.